



NLP Chatbot Development using Dialogflow

**Final Report (Software Requirements
Specification, Dialogflow Intent Structure &
Design Document)**

[Eatz Ai - Restaurant Management System]

Supervisor : Abdullah Qamar (abdullah.qamar@vu.edu.pk)

Group ID : F24PROJECT7B8AC (BC200417404)

TABLE OF CONTENTS

Sr. No.	Section Title
	Title Page
	Revision History
	Abstract
1.	Introduction
2.	Scope of the Project
3.	Project Overview
4.	Scope Boundaries
5.	Functional Requirements
6.	Non-Functional Requirements
7.	Assumptions and Constraints
8.	Use Case Diagrams
9.	Usage Scenarios / Use Cases
10.	Dialogflow Intent Structure
11.	System Architecture
12.	Technology Stack
13.	Entity Relationship Diagram (ERD)
14.	Sequence Diagrams
15.	Class Diagram
16.	Database Design
17.	Interface Design

18.	API Design
19.	Security Considerations
20.	Testing Plan and Test Cases
21.	Development Methodology
22.	Work Plan / Schedule
23.	References
24.	Appendices

Final Report: EatzaI – Restaurant Chatbot System

A Modern Solution for Effortless Ordering and Dining

Title Page

- Project Name: EatzaI – Restaurant Chatbot System
- Version: 1.0
- Group ID: F24PROJECT7B8AC
- Supervisor Name: Abdullah Qamar (abdullah.qamar@vu.edu.pk)
- Team Member(s): [Name : Syed Yahya Masood & Student IDs : BC200417404]
- Date: [Submission Date]

Revision History

Date	Version	Description	Author(s)
15/11/2024	1.0	Initial release: Software Requirements Specification (SRS) completed and submitted.	Bc200417404
27/02/2025	1.1	Added detailed Design Documents including architecture diagrams, ERD, and test cases.	Bc200417404

Abstract

EatzAI is an AI-powered restaurant management system that streamlines customer service through a conversational chatbot built with Google Dialogflow and a Laravel-based backend. The system consists of two distinct web portals: a **User Portal** for customers and an **Admin Portal** for restaurant staff.

Customers interact with the chatbot via web or mobile to place orders, reserve tables, browse the menu, and ask common questions—all using natural language. Upon registering, users gain access to the **User Portal**, where they can manage their profile, and view a detailed history of their past orders and reservations. This portal is a personalised dashboard linked directly to the chatbot's backend, ensuring all user activity is stored and accessible in real-time.

The **Admin Portal** allows staff to manage operational data such as menu items, orders, reservations, and customer FAQs. While the chatbot logic is maintained externally via Dialogflow, the admin interface handles all the supporting data that powers chatbot responses. By separating user and admin responsibilities and combining NLP-driven conversation with structured backend data, EatzAI offers a scalable, secure, and efficient solution for modern restaurant operations.

1. Introduction

EatzAI is an intelligent restaurant management system designed to automate routine customer interactions through a chatbot powered by **Google Dialogflow** and a robust **Laravel** backend. The project addresses the growing demand for fast, convenient, and personalized digital services in the restaurant industry. It streamlines key operations such as food ordering, table reservations, menu browsing, and responding to frequently asked questions using natural language conversations.

The system is built around a two-portal architecture: a **User Portal** and an **Admin Portal**. Through the User Portal, registered customers can manage their profile information through the User Portal, view detailed logs of their previous orders and reservations, and benefit from a personalized experience. The chatbot interface allows users to perform real-time actions like placing an order or reserving a table directly via conversation, offering both convenience and efficiency.

The Admin Portal enables restaurant staff to manage operational data such as menu items, orders, reservations, and frequently asked questions. While chatbot logic and intent flows are managed entirely within Dialogflow, the admin interface maintains and synchronizes the content that powers the chatbot's responses. This separation ensures maintainability, security, and clarity in system responsibilities.

EatzAI uses **Dialogflow's NLP capabilities** to identify user intents, while **Laravel APIs** handle business logic and database operations. The chatbot supports multi-turn conversations, contextual replies, and scalable architecture, ensuring the system is suitable for small to mid-sized restaurants.

By automating customer service interactions and providing real-time access to operational data, EatzAI enhances both customer satisfaction and staff productivity. It offers a future-ready solution that aligns with modern expectations for smart, AI-driven restaurant service platforms.

2. Scope of the Project

The scope of the EatzaI project is to design and implement an NLP-based restaurant management system that automates routine customer interactions using a chatbot powered by **Google Dialogflow**, with backend services developed in **Laravel**. The system aims to improve both the customer experience and operational efficiency by reducing manual workload, eliminating delays, and enabling real-time engagement through intelligent conversation.

EatzaI focuses on enabling customers to interact naturally with the restaurant system to perform tasks such as **placing food orders, making table reservations, browsing menu items, tracking their order status, and receiving answers to frequently asked questions**. All conversational interactions are handled by Dialogflow, while Laravel APIs manage data flow, logging, and integration with the backend database.

The system includes two portals:

- A **User Portal** where registered users can view their order and reservation logs, track order status, and manage their profile.
- An **Admin Portal** where restaurant staff can manage menu items, incoming orders, reservation records, and FAQs. Chatbot logic and conversational design remain within Dialogflow, while the Admin Portal provides the operational control interface.

The core functionalities within the project scope include:

- **Table Reservations:** Booking tables with dynamic date, time, and guest count input.
- **Menu Browsing:** Viewing categorized menu items with descriptions and prices.
- **Order Placement:** Adding food items to an order with quantity and customizations.
- **Order Tracking:** Allowing users to check the real-time status of their orders (e.g., confirmed, in preparation, ready).
- **FAQ Handling:** Responding to frequently asked questions like operating hours, contact details, and policies.
- **Customer Support:** Guiding users with automated help and escalating to human assistance when necessary.

By delivering these features through a conversational interface, EatzaI offers a modern, scalable, and user-friendly system that meets the evolving needs of both customers and restaurant operators.

3. Project Overview

The EatzaI project is a modern, AI-driven solution aimed at transforming restaurant service delivery through the integration of natural language processing and web-based management tools. At its core, EatzaI employs a chatbot built using **Google Dialogflow**, which interprets and processes user inputs in natural language and generates intelligent, context-aware responses. This chatbot acts as a virtual assistant, capable of handling restaurant-specific tasks such as **table reservations, order placement, menu exploration, order tracking, and FAQ handling**.

The chatbot is connected to a custom **Laravel-based backend**, which provides all supporting services, including API endpoints for retrieving and storing data related to menu items, orders, reservations, and customer information. This integration enables real-time synchronization between the chatbot interface and the system database, ensuring that users receive up-to-date responses based on live data managed by restaurant staff.

EatzaI is structured around a **dual-portal architecture**, offering separate interfaces for customers and administrators:

- The **User Portal** enables registered users to view and manage their personal information, order history, and reservation logs, and track the status of ongoing orders. While most user actions can be performed via the chatbot, the portal offers a structured interface for data access and account management.
- The **Admin Portal** allows restaurant staff to manage menu categories, food items, reservations, customer orders, and FAQs. It serves as the operational backend of the system, providing restaurant teams with full control over the data that supports chatbot responses. However, the conversational logic and intent management remain centralized within Dialogflow for modularity and clarity.

By combining real-time interaction with intelligent automation, EatzaI delivers a **seamless and engaging user experience**. The chatbot's ability to handle multi-turn conversations and follow contextual flows ensures a natural interaction process for customers, while the backend provides secure and efficient data handling. This design not only enhances customer convenience but also reduces the dependency on human staff for routine inquiries and tasks, leading to improved operational efficiency.

EatzaI is tailored for scalability and flexibility, making it suitable for small to mid-sized restaurants looking to modernize their service channels. With its role-based structure, RESTful API design, and responsive interfaces, the system stands as a robust digital solution to meet the evolving demands of the food service industry.

4. Scope Boundaries

The EatzaI system has been designed to address specific, well-defined functions within a restaurant environment. To ensure focus, maintainability, and delivery efficiency, the scope of this project is intentionally constrained to standard customer service operations. While the chatbot system is flexible and extensible, the following boundaries define what is **included** and **excluded** in the current version of the system:

Included in Scope

- Chatbot-based interaction for ordering, reservations, and FAQs.
- User portal for managing profiles and viewing order/reservation history.
- Admin portal for managing menu items, reservations, orders, customers, and FAQs.
- Real-time synchronization between chatbot and backend database.
- Order tracking and menu browsing through a conversational interface.

Out of Scope

- Online payment processing and payment gateway integration.
- Loyalty programs, reward points, and promotional campaigns.
- Integration with third-party delivery or POS systems.
- Handling complex complaints or personalized service issues beyond FAQ handling.
- Multilingual support (current implementation is English-only).

These boundaries help ensure that the project remains focused on delivering a stable and effective system that can be extended in future iterations.

5. Functional Requirements

The following are the key functional requirements of the EatzaI system:

FR No.	Requirement Description
FR1	The system shall allow users to place food orders via chatbot.
FR2	The system shall enable users to browse the restaurant menu through categorized queries.
FR3	The system shall allow users to reserve a table by specifying the date, time, name, and party size.
FR4	The system shall track and display the real-time status of a user's order.
FR5	The system shall answer frequently asked questions using predefined responses.
FR6	The system shall allow registered users to view their order and reservation history.
FR7	The admin shall be able to manage menu items, orders, and reservations through a secure portal.
FR8	The chatbot shall integrate with Laravel APIs for data storage and retrieval.
FR9	The admin shall be able to view and manage registered customer profiles.

6. Non-Functional Requirements

The non-functional requirements define how the EatzAI system should perform and operate. These include system behaviour, quality attributes, and constraints.

NFR No.	Requirement Description
NFR1	The chatbot shall respond to user queries within 10–30 seconds.
NFR2	The system shall be able to support at least 50 concurrent users.
NFR3	The chatbot and portals shall be accessible 24/7 through web and mobile platforms.
NFR4	All user data, such as orders and reservations, shall be stored securely in the backend database.
NFR5	The system shall implement basic security controls including user authentication for portal access.
NFR6	The system shall use modular and maintainable code architecture to allow future expansion.
NFR7	The admin portal shall be protected by role-based access control.
NFR8	The chatbot shall handle both single-turn and multi-turn conversations using Dialogflow contexts.

7. Assumptions and Constraints

This section outlines the assumptions made during the development of the EatzaAI system and the constraints that have influenced its design and implementation. Understanding these factors helps define the system's operational boundaries and ensures realistic stakeholder expectations.

7.1 Assumptions

The following assumptions were made during the planning and development phases:

- **A1:** Users will have access to the internet and a modern browser to interact with the chatbot.
- **A2:** The chatbot will primarily communicate in **English**; multilingual support is not part of the current release.
- **A3:** Restaurant staff will manage menu items, orders, FAQs, and reservation data through the **Admin Portal**.
- **A4:** Users will access their personal data (orders, reservations, profile) through the **User Portal**, which is linked to chatbot interactions.
- **A5:** Users are expected to follow typical conversational patterns (e.g., "I want to order a pizza") for accurate intent recognition.
- **A6:** The restaurant will provide consistent and structured data (menu, categories, FAQs) required for chatbot training and backend integration.
- **A7:** The backend (Laravel) and chatbot (Dialogflow) servers will remain available during operational hours to ensure seamless response handling.

7.2 Constraints

The following constraints have been considered and addressed in the system design:

- **C1:** The system does **not support online payments** in the current version.
- **C2:** The chatbot logic is **exclusively managed within Dialogflow**, separate from Laravel backend.
- **C3:** The system supports **web and mobile interfaces** only; no voice-based or kiosk interactions are implemented.

- **C4:** The chatbot is dependent on **Dialogflow's uptime and service availability**.
- **C5:** User authentication and profile management are required to access personalized content in the portal.
- **C6:** The backend uses **MySQL** as the database; integration with other DBMS platforms is not included.
- **C7:** The system handles **basic user queries and support**; complex customer service issues must be escalated manually outside the chatbot.

This section sets clear boundaries around what the system is expected to do, under which conditions it operates, and which limitations were accounted for during development.

8. Use Case Diagrams

Use Case Diagrams visually represent the interactions between users (actors) and the system. Eatza! defines two primary actors — **User (Customer)** and **Admin (Staff)**. Each diagram shows the core functions accessible to each role through either the chatbot or the web portals.

User Use Case Diagram Includes:

- Browse Menu
- Place Order
- Track Order
- Reserve Table
- Ask FAQs
- View Order History
- View Reservation History
- Manage Profile

Admin Use Case Diagram Includes:

- Manage Menu
- Manage Orders
- Manage Reservations
- Update Order / Reservation Status
- Manage FAQs
- Manage Customer Profiles

Figure 8.1: User Use Case Diagram

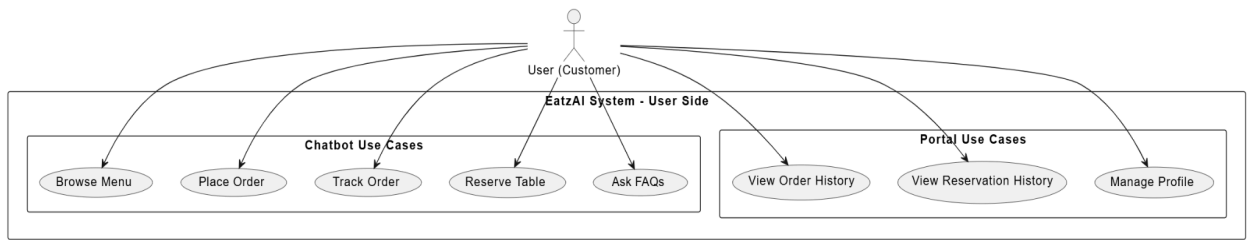
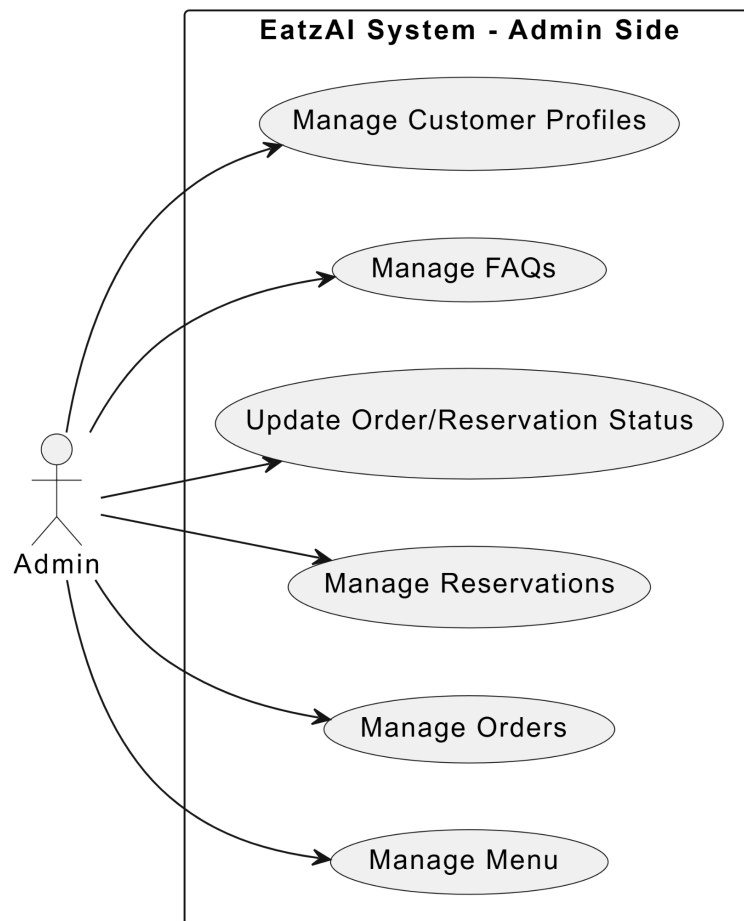


Figure 8.2: Admin Use Case Diagram



9. Usage Scenarios / Use Cases

This section provides textual descriptions of how users and administrators interact with the EatzaI system in real-world scenarios. Each use case includes the actors involved, preconditions, main flow, and expected outcomes.

Use Case 1: Place Food Order via Chatbot

- **Actor:** User (Customer)
- **Preconditions:** User accesses chatbot interface. The menu must be available.
- **Main Flow:**
 1. The user launches the chatbot.
 2. The user requests to view the menu.
 3. The chatbot displays categorized menu items.
 4. The user selects food items and quantity.
 5. The chatbot confirms the order and places it via the backend.
 6. The user receives the confirmation and order ID.
- **Postconditions:** The order is logged in the database and visible in the user portal.

Use Case 2: Reserve a Table

- **Actor:** User (Customer)
- **Preconditions:** Reservation slots are available.
- **Main Flow:**
 1. The user initiates a reservation through the chatbot.
 2. The user provides the name, date, time, and number of guests.
 3. Chatbot confirms and saves reservations to the database.
 4. Confirmation is shown with the reservation ID.
- **Postconditions:** Reservation is recorded and shown in the user portal history.

Use Case 3: Track an Order

- **Actor:** User (Customer)
- **Preconditions:** Order must exist and have a valid status.
- **Main Flow:**
 1. The user asks the chatbot to track their order.
 2. Chatbot retrieves order status from the backend.
 3. Chatbot replies with real-time status (e.g., Preparing, Ready, Delivered).
- **Postconditions:** Order status is displayed to the user.

Use Case 4: Manage Customer Profile

- **Actor:** Admin
- **Preconditions:** Admin must be authenticated.
- **Main Flow:**
 1. Admin logs into admin portal.
 2. Admin navigates to the customer management section.
 3. Admin views customer list and selects a profile.
 4. Admin updates profile details or disables account if needed.
- **Postconditions:** Updated user profile data is stored in a database.

Use Case 5: Manage Menu Items

- **Actor:** Admin
- **Preconditions:** Admin must be authenticated.
- **Main Flow:**
 1. Admin logs in and opens the menu management module.
 2. Admin adds, updates, or deletes menu items.

3. Changes are reflected in the chatbot's menu via API sync.

- **Postconditions:** Menu data is updated in the backend and shown to users.

Summary Table of Use Cases

Actor	Use Case	Description
User	Browse Menu	View available menu categories and items.
User	Place Order	Select items and confirm the order via chatbot.
User	Track Order	Check the status of an ongoing order.
User	Reserve Table	Book a table for a specified date, time, and party size.
User	Ask FAQs	Ask general questions such as hours, location, etc.
User	View Order History	See past orders via the user portal.
User	View Reservation History	View booking history.
User	Manage Profile	Update personal details like name or contact info.
Admin	Manage Menu	Add, remove, or modify food items.
Admin	Manage Orders	View all incoming orders and their details.
Admin	Manage Reservations	View or cancel table bookings.
Admin	Update Order/Reservation	Update the status of orders or reservations.
Admin	Manage FAQs	Modify responses for common user inquiries.
Admin	Manage Customer	Allows the admin to view, update, or deactivate customer accounts.

10. Dialogflow Intent Structure

The Dialogflow Intent Structure defines the conversational logic and natural language understanding (NLU) workflows for the EatzaI restaurant chatbot. It specifies how user inputs are interpreted and responded to, using intents, entities, and context handling.

Dialogflow enables the system to support fluid, human-like interactions while maintaining accurate and goal-oriented behaviour. Each intent corresponds to a specific user intention and is trained with multiple example phrases (training data) for improved recognition.

10.1 Intent Classification

Intents are categorised into four main functional groups:

A. Menu Navigation

Intent Name	Purpose
menu.show	Displays all available menu categories
menu.category	Shows items under a specific category

B. Order Management

Intent Name	Purpose
order.add	Add item(s) to an ongoing order
order.remove	Remove item(s) from the current order
place.order	Finalize and submit the order
checkout.order	Provide order summary before confirmation
order.status	Retrieve and display order status
cancel.order	Cancel the current or a specific order

C. Reservation Management

Intent Name	Purpose
reservation.create	Collects data to make a new table reservation
reservation.status	Displays details of an existing reservation

D. General and Support

Intent Name	Purpose
faq.inquiry	Respond to frequently asked questions
customer.details	Collect or update user profile information
Default Welcome	Greet the user and initiate interaction
Default Fallback	Handle unrecognized inputs

10.2 Entities Used

Entity Name	Purpose / Values
@menu-category	drinks, desserts, main course, starters
@item	pizza, cola, burger, cake
@quantity	one, two, 1, 2
@faq-topic	hours, payment methods, address, allergy info
@status-type	tracking, status, progress
@datetime	tomorrow at 8PM, 25th Dec, next Saturday
@party-size	2 people, 4 persons
@name, @email, @phone	For collecting customer profile details
@meal-type	Ask the customer for lunch or dinner

10.3 Context Management

Dialogflow uses **input and output contexts** to manage multi-turn conversations and maintain the state between steps.

Intent	Input Context	Output Context
menu.category	awaiting_menu_category	awaiting_item_selection
order.add	awaiting_item_selection	order-building
order.status	—	—
reservation.create	—	awaiting_confirmation

This ensures that, for example, after a user views the drinks menu, the chatbot knows the next message like “add cola” refers to an item selection, not a new topic.

10.4 Integration with Backend (Laravel)

Dialogflow uses webhook fulfilment to trigger Laravel backend APIs. These APIs handle:

- Real-time menu updates
- Order/reservation creation and tracking
- Data storage and retrieval for user profiles

This architecture allows Dialogflow to act as the conversational front end, while Laravel handles all business logic and data management behind the scenes.

11. System Architecture

The system architecture of EatzAI is designed using a **modular, layered approach**, ensuring scalability, maintainability, and seamless integration between conversational AI and backend services. The system consists of four primary components:

1. **Chatbot Interface (Frontend)**
2. **NLP Engine (Google Dialogflow)**
3. **Backend Server (Laravel)**
4. **Database (MySQL)**

11.1 Architectural Overview

The EatzAI system follows a **client-server model** with integrated RESTful communication between the chatbot and the backend. Below is a breakdown of each layer:

5. **Chatbot Interface (Web/Mobile)**
 - Acts as the user-facing interface embedded on the restaurant's website or app.
 - Accepts natural language input from the user.
 - Displays conversational responses from Dialogflow in real-time.
6. **NLP Engine (Google Dialogflow)**
 - Interprets user messages and maps them to corresponding intents.
 - Uses contexts and entities to manage multi-turn conversations.
 - Forwards intent data to Laravel backend via webhooks.
7. **Backend (Laravel)**
 - Acts as the core business logic processor.
 - Provides REST APIs to Dialogflow and both portals.
 - Authenticates admin/user access and enforces role-based access control.
 - Handles operations such as:
 - ❖ Menu retrieval

- ❖ Order/reservation creation
- ❖ Order tracking
- ❖ Profile management

8. Database (MySQL)

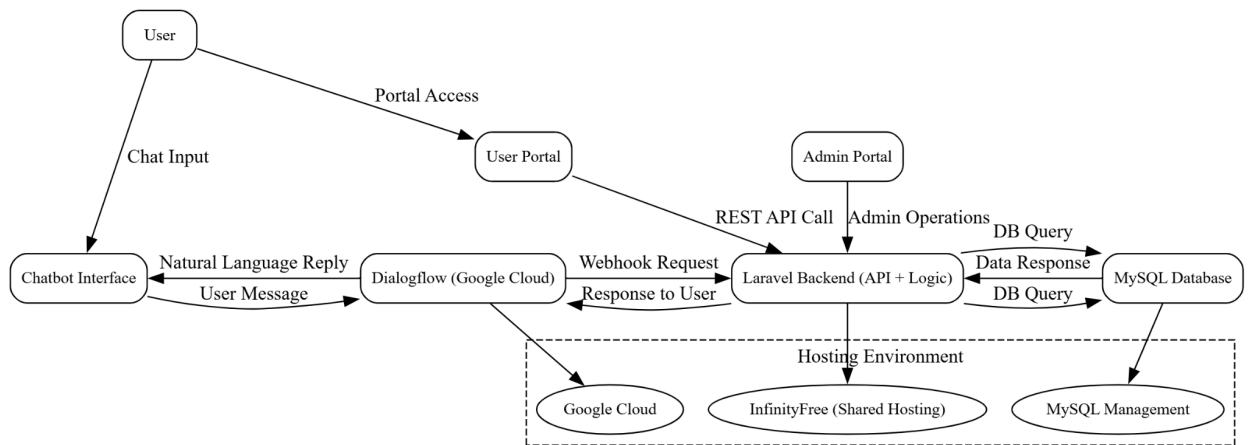
- Stores all persistent data:
 - ❖ Users
 - ❖ Orders
 - ❖ Reservations
 - ❖ Menu items
 - ❖ Admin-defined FAQs

11.2 Interaction Flow

9. The user types a message in the chatbot interface.
10. Dialogflow processes the message and detects the intent.
11. If required, Dialogflow calls Laravel backend APIs via a webhook.
12. Laravel processes the request interacts with the database, and returns a response.
13. Dialogflow uses the webhook response to continue the conversation with the user.

11.3 Architecture Diagram

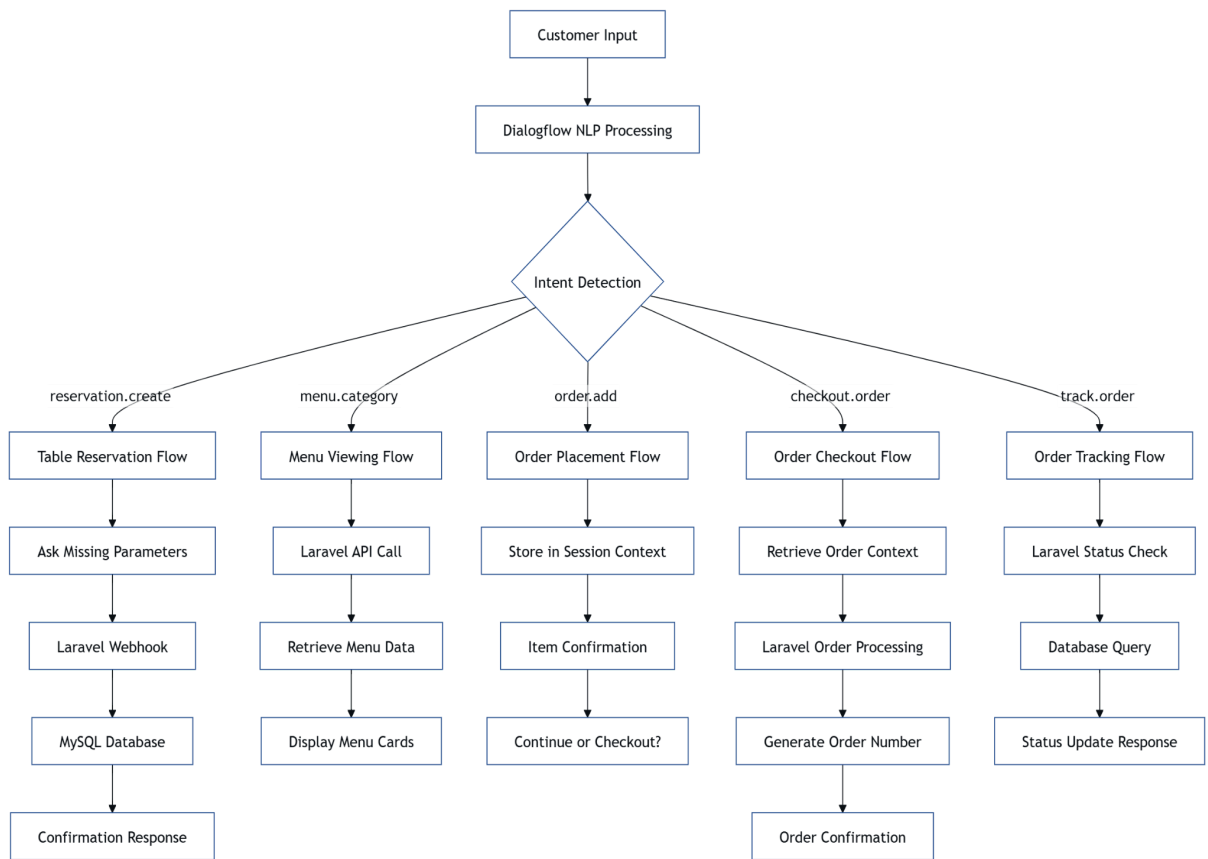
[Figure 11.1 – System Architecture Diagram here]



This diagram should illustrate:

- User → Chatbot Interface → Dialogflow → Laravel Backend → MySQL Database
- Admin/User Portals accessing Laravel backend directly.

[Figure 11.2 – System Architecture chatbot Diagram here]



12. Technology Stack

The EatzaI system is developed using a carefully selected set of modern web technologies and tools, ensuring a responsive, secure, and scalable solution for restaurant chatbot automation. The technology stack is structured across four main layers: frontend, backend, NLP engine, and database.

12.1 Frontend

Technology	Purpose
HTML5 / CSS3	Markup and styling of the User and Admin portals.
JavaScript	Dynamic interactivity and user input handling.
Bootstrap 5	Responsive layout for multi-device compatibility.
Chatbot Widget	Embeds Dialogflow chatbot into the user interface.

12.2 Backend

Technology	Purpose
Laravel 10 (PHP)	Backend framework for REST APIs, business logic, authentication, and user management.
Custom Role-Based Access (DB-driven)	Role distinction between users and admins using database conditions (without external packages).
Laravel Auth	Authentication and session handling for portal users.
Composer	PHP package management.

12.3 NLP Engine

Technology	Purpose
Google Dialogflow ES	Processes natural language detects intents, and triggers appropriate actions.
Webhook Integration	Sends intent data to Laravel for dynamic responses and data operations.

12.4 Database

Technology	Purpose
MySQL 8.0	Relational data storage for users, orders, reservations, and menu items.
phpMyAdmin	Graphical interface for managing MySQL tables.

12.5 Hosting & Deployment

Technology	Purpose
XAMPP / Laravel	Local development environment.
cPanel / Shared Hosting	Hosts Laravel app and MySQL database for live deployment.
Google Cloud / Firebase (Dialogflow)	Hosts the Dialogflow agent for chatbot operations.

12.6 Development Tools

Tool	Purpose
VS Code	Source code editor.
Git & GitHub	Version control and collaboration.
Postman	API testing and debugging.
draw.io	Diagramming (ERD, sequence, class diagrams).

12.7 Deployment Environment

The EatzaI system is deployed on **InfinityFree**, a free web hosting platform that supports PHP and MySQL. The live application can be accessed at :

<https://eatza-ai.infinityfreeapp.com/?i=1>

Key Deployment Details:

- **Hosting Provider:** InfinityFree
- **Server Stack:** Apache, PHP 7.x, MySQL
- **Deployment Method:** Files uploaded via InfinityFree's File Manager
- **Database Management:** phpMyAdmin provided by InfinityFree
- **SSL Support:** Free SSL certificate provided by InfinityFree

Deployment Considerations:

- **File Updates:** Changes to PHP, CSS, or JavaScript files may not reflect immediately due to browser caching. To ensure updates are visible:
 - Clear your browser cache after making changes.
 - Use versioning in your file references (e.g., style.css?v=2) to force the browser to load the latest version.

- Ensure that files are uploaded to the correct directories, typically the htdocs folder.
- Database Configuration: Ensure that your database connection settings in the Laravel .env file match the credentials provided by InfinityFree.
- **Error Handling:** Enable Laravel's debug mode during development to assist in identifying and resolving issues. Remember to disable debug mode in the production environment to prevent exposure to sensitive information.

13. Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) provides a logical overview of the data structure used in the EatzaI system. It visually represents the major entities in the system and the relationships between them, helping developers, stakeholders, and testers understand how data is stored and interconnected.

13.1 Purpose

The ERD is essential in ensuring that the backend Laravel database schema is well-organized, normalized, and optimized for relational operations. It also guides the creation of Laravel models, migrations, and API relationships.

13.2 Key Entities

The primary entities in the EatzaI system include:

Entity	Description
User	Represents registered customers using the chatbot or portal.
Admin	Represents staff members who manage the system.
Order	Contains customer order data, including items and status.
Order_Item	Links orders to specific menu items and quantities.
Reservation	Stores reservation details like date, time, and party size.
Menu_Item	Represents individual food or drink items.
Category	Classifies menu items into groups like drinks or desserts.
FAQ	Stores common customer questions and answers.

13.3 Relationships

- **User – Order:** One-to-Many (A user can place multiple orders)
- **User – Reservation:** One-to-Many (A user can make multiple reservations)
- **Order – Order_Item:** One-to-Many (Each order contains multiple items)
- **Menu_Item – Order_Item:** One-to-Many (Each item can appear in many orders)
- **Category – Menu_Item:** One-to-Many (Each category has multiple items)
- **Admin – FAQ:** One-to-Many (Admin can manage multiple FAQs)

13.4 Diagram

[Figure 13.1 – Entity Relationship Diagram]

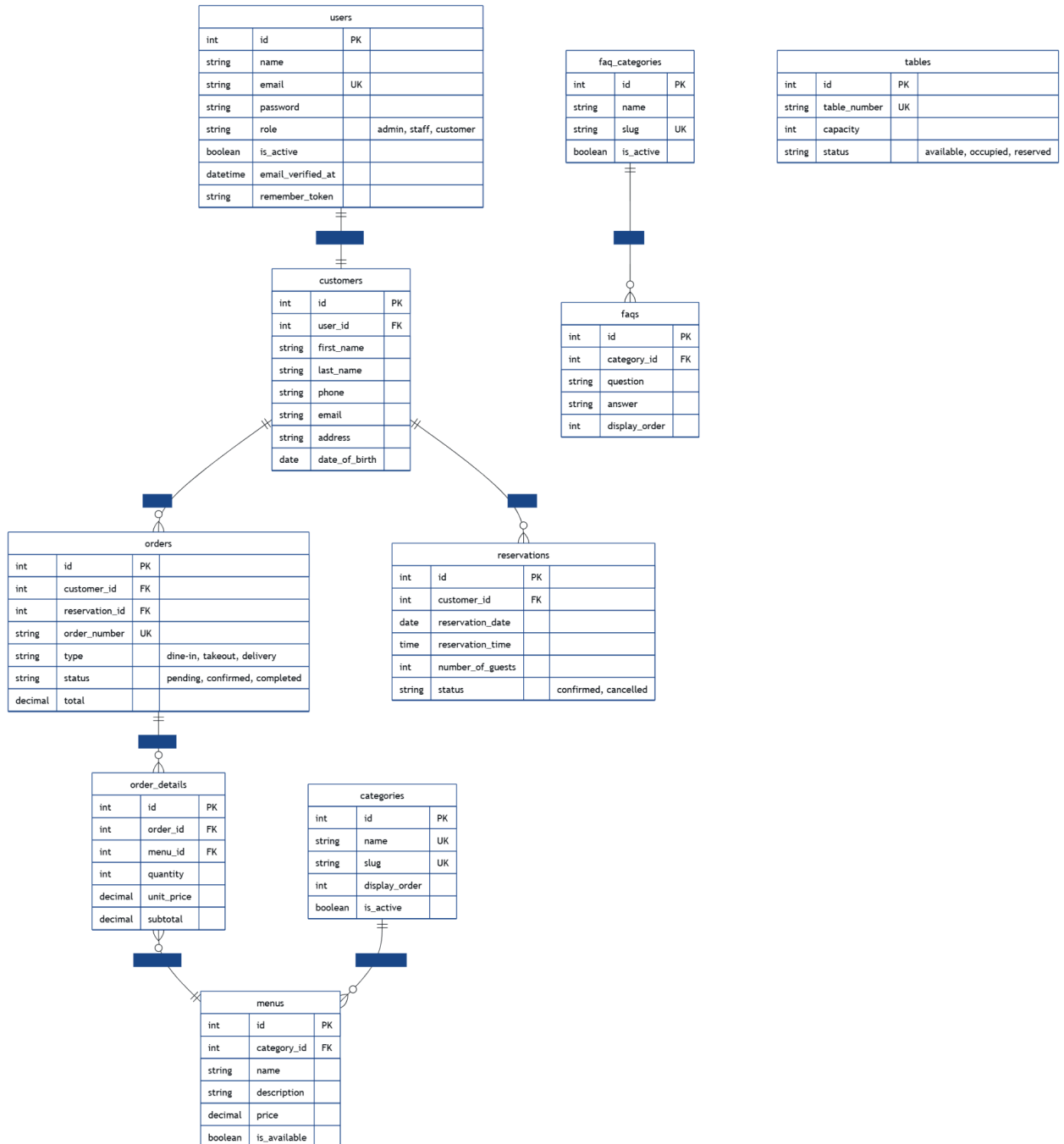


Figure 13.1: ERD depicting the relational structure of users, orders, reservations, and menu items.

14. Sequence Diagrams

Sequence diagrams are a type of UML (Unified Modeling Language) diagram that describe how system components interact with each other over time. These diagrams illustrate the order of messages exchanged between various objects in the system during the execution of specific use cases.

In the EatzAI system, sequence diagrams are used to visualize the communication flow between the **user (via chatbot interface)**, **Dialogflow**, **Laravel backend**, and the **MySQL database** for key user scenarios.

14.1 Purpose

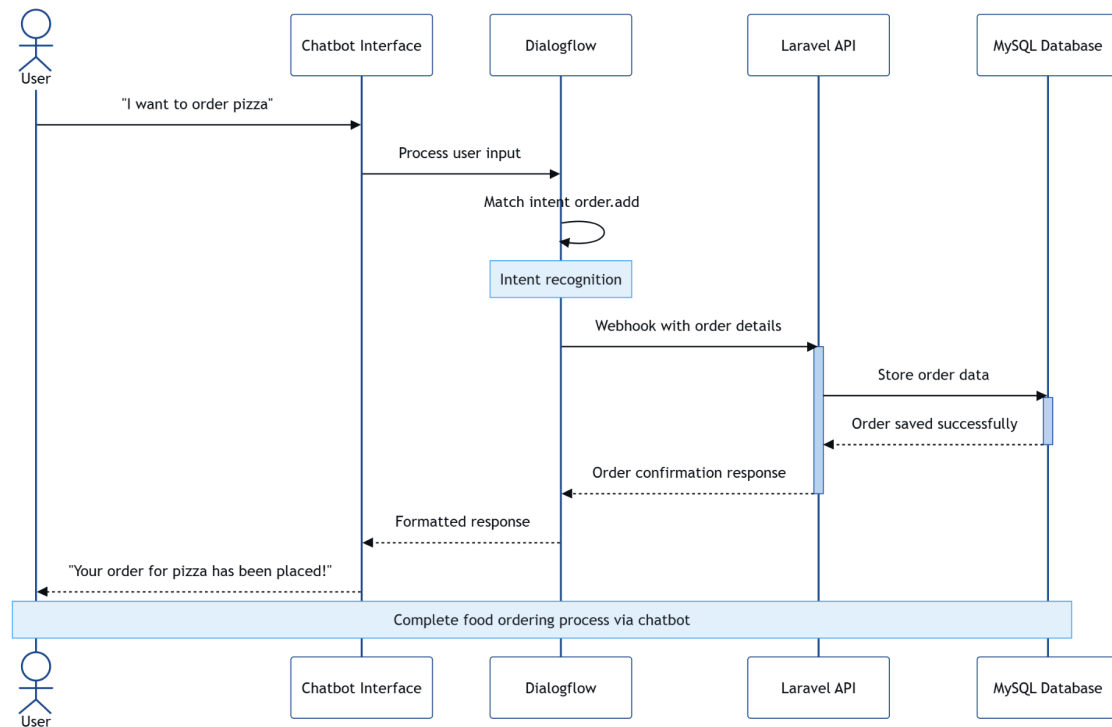
The main purpose of these diagrams is to:

- Understand system behavior across use cases.
- Illustrate how chatbot messages trigger backend operations.
- Provide a visual reference for developers and testers.
- Ensure consistency in handling user requests and responses.

14.2 Key Sequence Diagrams

14.2.1 Place Food Order via Chatbot

[Figure 14.1 – Sequence Diagram: Place Order]



Actors Involved:

- User
- Chatbot Interface
- Dialogflow
- Laravel API (Backend)
- MySQL Database

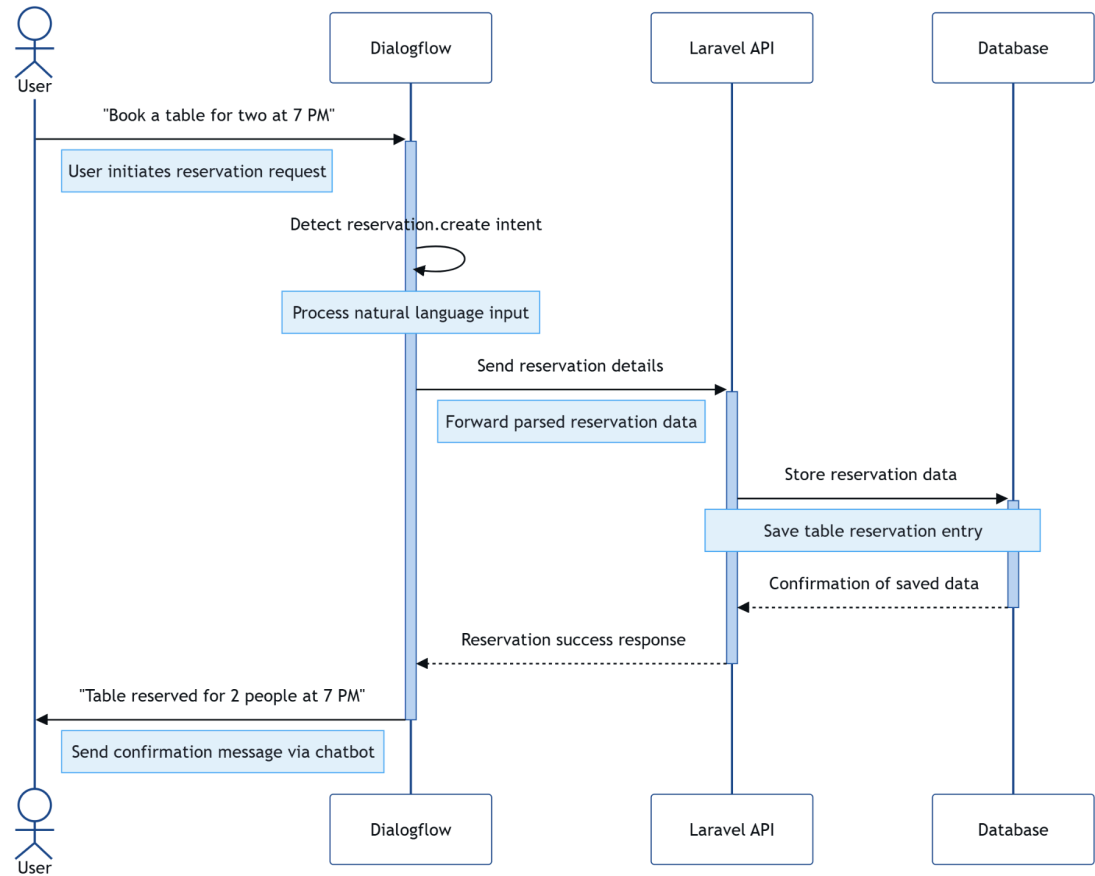
Flow:

1. User types "I want to order pizza".
2. Dialogflow matches the intent order.add.
3. Webhook sends order details to Laravel API.

4. Laravel stores order and responds with confirmation.
5. Chatbot replies with “Your order for pizza has been placed!”

14.2.2 Reserve a Table

[Figure 14.2 – Sequence Diagram: Reserve Table]



Actors:

- User
- Dialogflow
- Laravel API

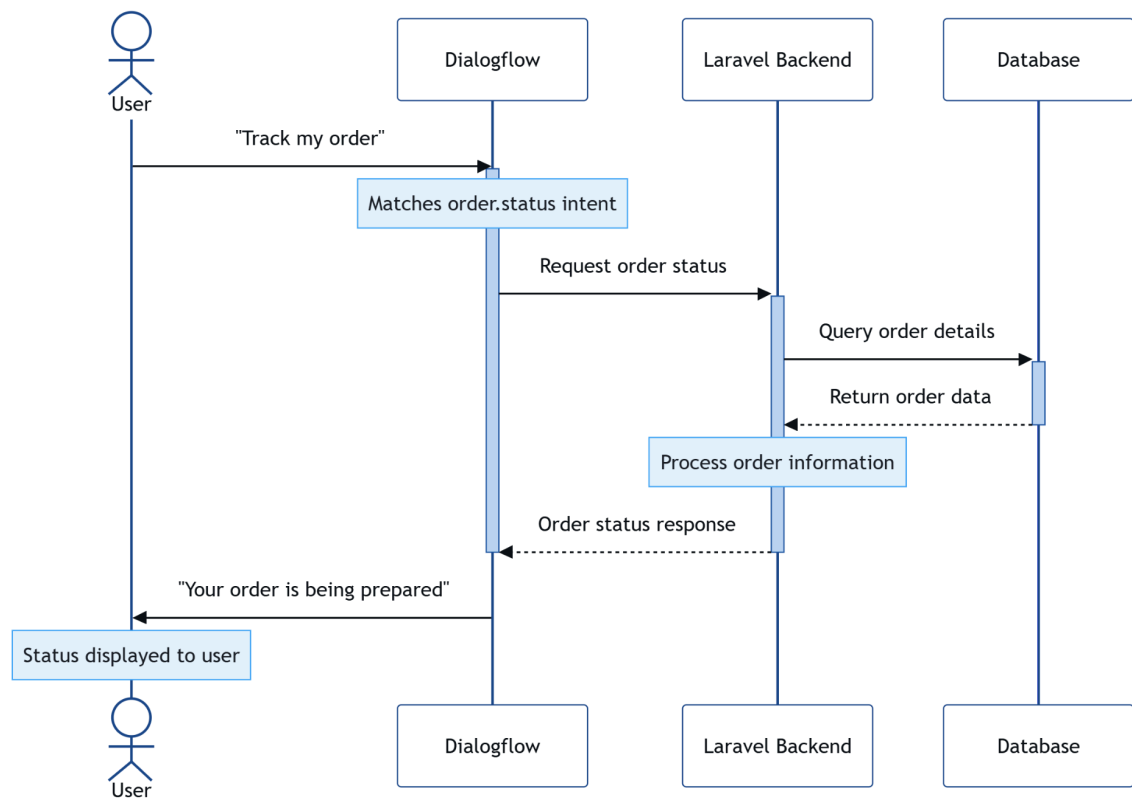
- Database

Flow:

1. User: "Book a table for two at 7 PM".
2. Dialogflow detects reservation.create intent.
3. Laravel API stores reservation data.
4. The database saves the reservation entry.
5. Confirmation message sent back via chatbot.

14.2.3 Track an Order

[Figure 14.3 – Sequence Diagram: Track Order]



Actors:

- User
- Dialogflow
- Laravel Backend
- Database

Flow:

1. User: "Track my order".
2. Dialogflow matches `order.status` intent.
3. The backend fetches order status.
4. Status ("Your order is being prepared") returned and displayed.

14.3 Tools Used

- Draw.io / Lucidchart for visual modelling
- Standard UML notation
- Integrated with use case documentation

Sequence diagrams are an essential part of the system design. They provide a real-time view of user–system interactions and ensure alignment between development and chatbot logic. Including these diagrams in your report demonstrates your understanding of asynchronous communication and system flow.

15. Class Diagram

The Class Diagram is a fundamental part of object-oriented design. It visually represents the system's classes, attributes, methods, and the relationships between classes. In the EatzAI system, the class diagram helps developers understand how different objects (such as Users, Orders, Reservations, etc.) interact within the backend codebase built using Laravel.

15.1 Purpose

- To define the static structure of the system.
- To visualize how objects like Users, Orders, and Menu Items are modeled in the database and backend logic.
- To support maintainable and reusable code structure in Laravel.

15.2 Key Classes

The main classes represented in the system are:

Class	Attributes	Methods
User	id, name, email, password, role	register(), login(), updateProfile()
Order	id, user_id, total_price, status, created_at	placeOrder(), getStatus(), cancelOrder()
OrderItem	id, order_id, item_id, quantity, subtotal	calculateSubtotal()
Reservation	id, user_id, datetime, guests, status	createReservation(), cancelReservation()
MenuItem	id, name, description, price, category_id	getMenuByCategory(), updateItem()
Category	id, name	listCategories()
FAQ	id, question, answer, created_by (admin_id)	getAllFAQs(), updateFAQ()
Admin	id, name, email, password	login(), manageUsers(), manageContent()

15.3 Relationships Between Classes

- A **User** can place multiple **orders** and make **reservations**.
- An **Order** can have many **OrderItems**.
- An **OrderItem** is associated with one **MenuItem**.
- A **MenuItem** belongs to one **Category**.
- An **Admin** can manage multiple **FAQs** and view customer data.

15.4 Class Diagram Overview

[Figure 15.1 – Class Diagram for EatzaI System]

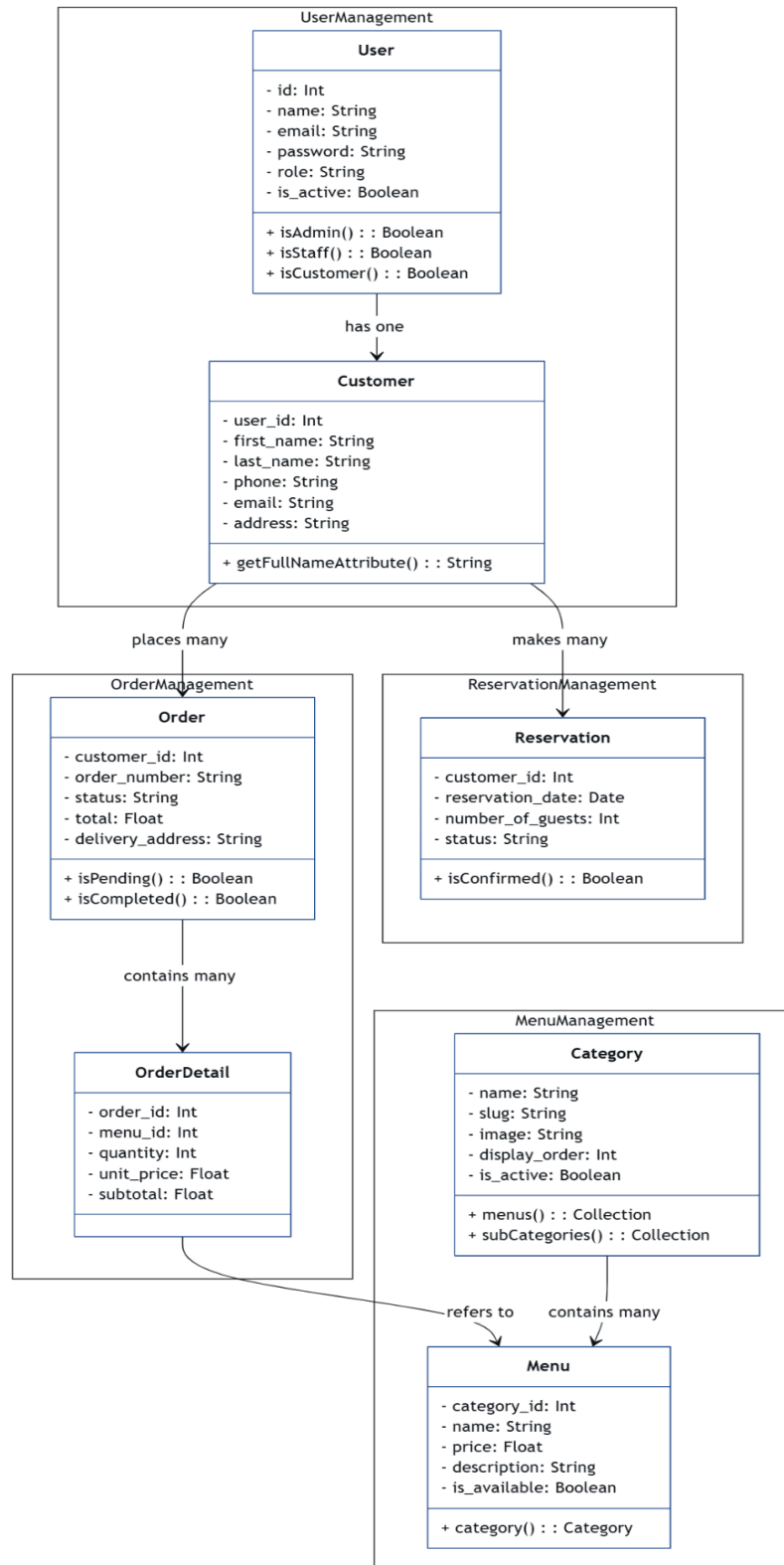


Figure 15.1: UML class diagram showing associations, multiplicities, and class members (attributes + methods).

The diagram should reflect:

- Inheritance if any (e.g., Admin and User as subclasses – optional if both are separated).
- Associations (one-to-many, many-to-one).
- Aggregation or composition where appropriate (e.g., Order–OrderItem).

15.5 Implementation Note

In Laravel, each class corresponds to an Eloquent model that interacts with the MySQL database. The class diagram helps ensure:

- Consistency between code and schema.
- Reusability of backend components.
- Easier debugging and scaling of the system.

16. Database Design

The database design of the EatzaI system is developed using a normalised relational structure to ensure data integrity, minimise redundancy, and support efficient querying for chatbot responses and portal operations. MySQL powers the backend, and the design aligns with the system's class structure and API functionality.

16.1 Purpose

The database serves as the central data store for all operations, including:

- User and admin authentication
- Order and reservation tracking
- Menu and category management
- FAQ storage
- Role-based access handling

It supports both chatbot-based operations (via API) and web portal activities for users and administrators.

16.2 Core Tables

Below are the key tables used in the EatzaI system:

16.3 Relationships Summary

- **One-to-Many:** A user can have many orders or reservations.
- **Many-to-Many (via order_items):** A single order can contain multiple menu items.
- **One-to-Many:** A category contains many menu items

16.4 Normalization

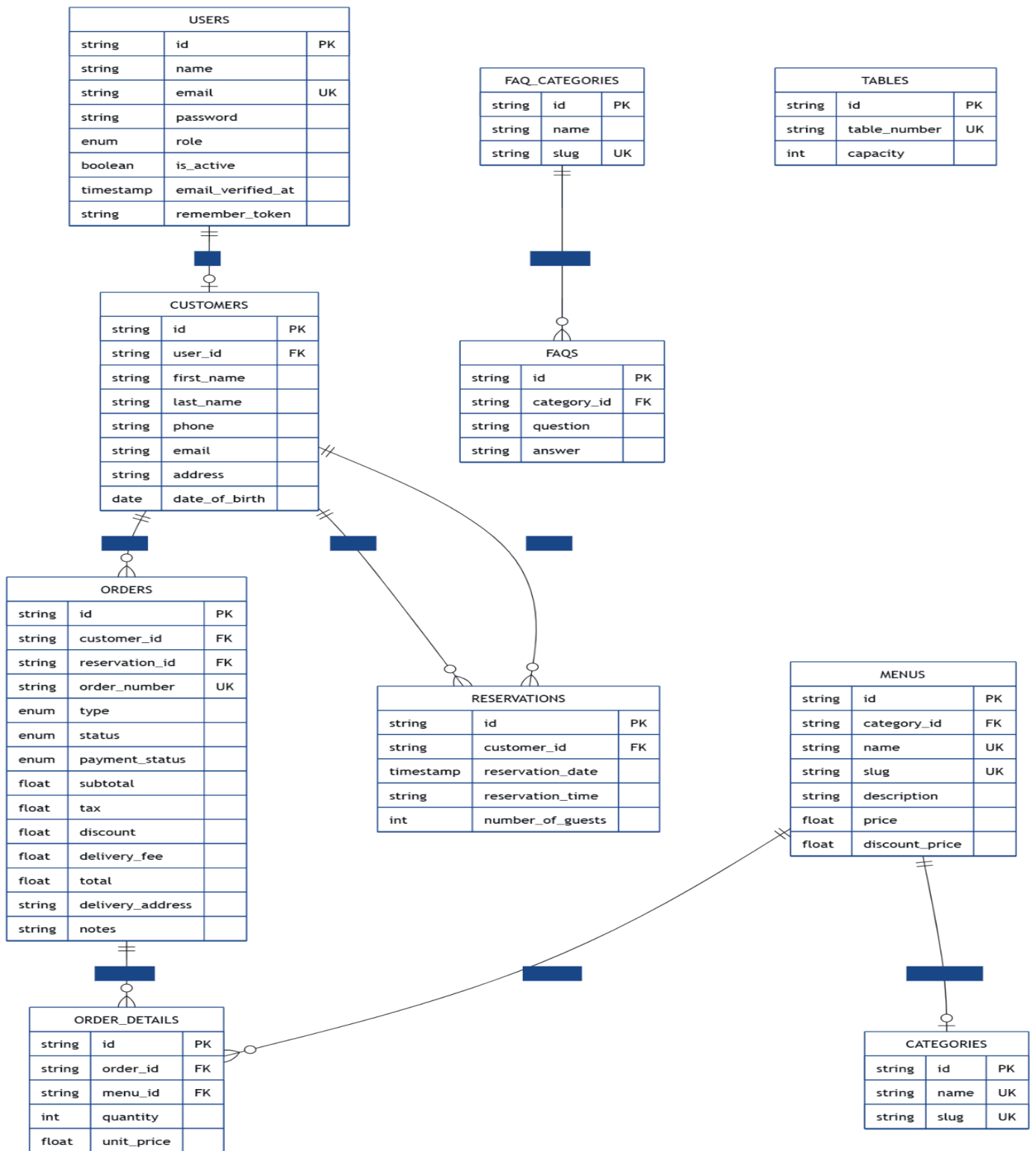
The database follows **3rd Normal Form (3NF)** to ensure:

- No duplicate data
- Proper foreign key constraints
- Efficient indexing for fast query execution

16.5 Tools Used

- **phpMyAdmin:** For managing and testing the MySQL schema
- **Laravel Migrations:** For schema creation and versioning
- **MySQL Workbench / draw.io:** For designing the ERD

[Figure 16 – Database Design for EatzAI System]



17. Interface Design

The interface design of the EatzaI system is focused on usability, clarity, and responsiveness. The system includes two distinct web-based portals — one for **Users** (customers) and one for **Admins** (restaurant staff) — along with the integrated **chatbot interface** powered by Dialogflow. Each interface has been designed to support the functional goals of its users with a clean layout, minimal learning curve, and easy navigation.

17.1 Design Goals

- Provide a **user-friendly experience** across all devices (desktop, tablet, mobile).
- Ensure that both **customers and staff** can perform their tasks efficiently.
- Maintain **UI consistency** across pages for intuitive navigation.
- Integrate the **chatbot interface** as an accessible assistant, not a distraction.
- Use **Bootstrap** to ensure responsiveness and standard UI components.

17.2 User Portal Screens

The User Portal allows registered users to interact with their data and view logs generated through chatbot activities.

Key Screens:

Screen	Functionality
Login/Register Page	User authentication and account creation
Dashboard	Welcome screen showing quick links to orders/reservations
My Orders	List of all past and current orders with status tracking
My Reservations	Booking history with reservation ID, date, and status

Profile Management	Edit name, email, password, contact info
Chatbot Interface	Embedded chatbox to interact with the Dialogflow agent

17.3 Admin Portal Screens

The Admin Portal is used by restaurant staff to manage operational data such as menu items, customer profiles, orders, and FAQs.

Key Screens:

Screen	Functionality
Admin Login Page	Secure admin authentication
Admin Dashboard	Overview of orders, reservations, and user activity
Menu Management	Add, edit, delete food categories and menu items
Order Management	View, update, and mark order statuses
Reservation Management	View and update table bookings
Customer Management	View and manage user profiles
FAQ Management	Add, edit, or delete chatbot FAQs

17.4 Chatbot Interface

The chatbot is embedded on the user-facing frontend and available across all major pages. It is positioned as a floating widget that expands upon interaction and supports:

- Real-time ordering
- FAQ inquiries

- Menu navigation
- Table reservations

The chatbot interface is styled to distinguish between user and bot messages, using different colors and avatars to represent each.

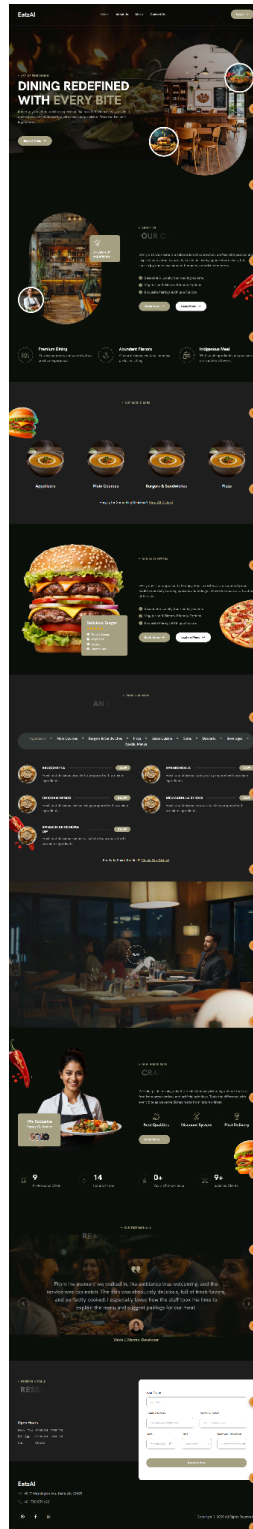
17.5 Tools Used for UI Design

- **HTML5, CSS3, Bootstrap 5:** Responsive layout and styling
- **Blade Templates (Laravel):** Dynamic rendering of content
- **JavaScript / jQuery:** Client-side interactivity
- **Dialogflow Web Messenger:** Integrated chatbot widget
- **Figma / draw.io (optional):** For UI mockup planning

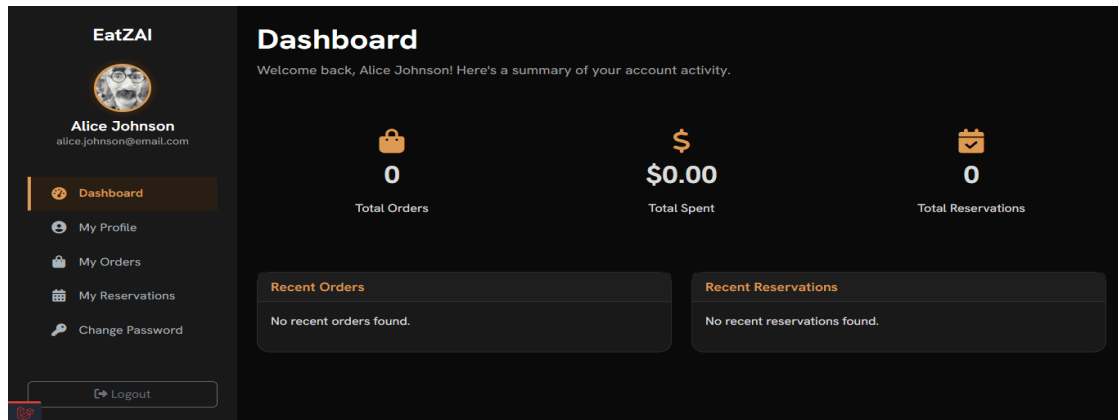
17.6 Accessibility Considerations

- Color contrast maintained for readability
- Responsive UI for mobile users
- Simple language and action buttons for non-technical users
- Tab-indexing and form validation for usability

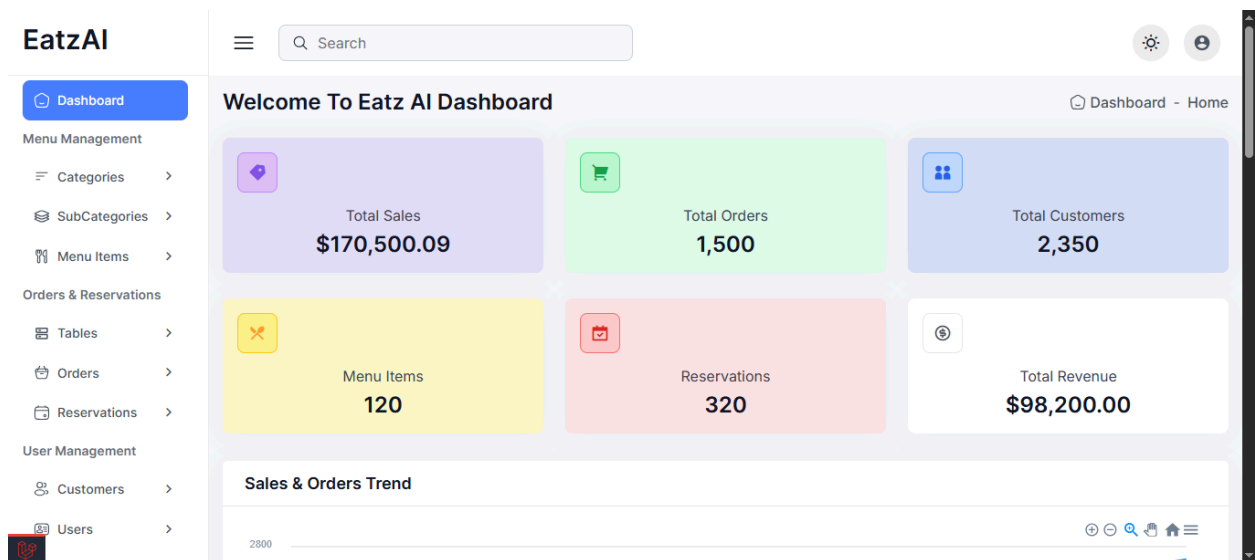
[Screenshots or mockups here if available, labelled as Figure 17.1, 17.2, etc.]



[Figure 17.1 – Website Page]



[Figure 17.2 – User Portal]



[Figure 17.3 – Admin Portal]

18. API Design

The EatzAI system uses a **RESTful API architecture** to enable seamless communication between the **Dialogflow chatbot**, **user/admin portals**, and the **Laravel backend**. These APIs are essential for handling data exchange, ensuring real-time updates, and maintaining consistency across the system.

18.1 Purpose of the API Layer

- Facilitate secure communication between Dialogflow and Laravel backend
- Allow the user portal and admin panel to access backend data via structured endpoints
- Separate the frontend and backend concerns for better scalability and modularity
- Provide consistent and standardized interaction points for third-party integration in the future

18.2 API Characteristics

Characteristic	Value
Architecture Style	RESTful
Authentication	Laravel Auth (Session / Token-based)
Response Format	JSON
Error Handling	Standard HTTP status codes + error JSON
Security	Middleware-based access control

Great! Here's your updated **Section 18.3: Key API Endpoints** content, replacing the original brief version with your **full professional, detailed version**—formatted and ready for use in your software documentation.

18.3 Key API Endpoints

EatzAI Restaurant Management System

Below is a comprehensive list of all API endpoints exposed by the Laravel backend, including both RESTful API routes and web-based endpoints that return JSON for AJAX interactions.

Authentication & User Management

Includes both API and web routes for registration, login, password reset, and user profile management.

Method	Endpoint	Description	Auth Required
POST	/api/login	API login for users/admins	No
POST	/api/register	API registration	No
GET	/api/user	Fetch logged-in user info (API)	Yes (Sanctum)
POST	/login	Web login	No
POST	/register	Web registration	No
POST	/logout	Web logout	Yes
GET	/forgot-password	Show forgot password form	No
POST	/forgot-password	Send reset link	No
GET	/reset-password/{token}	Show reset form	No
POST	/reset-password	Submit new password	No
GET	/user/profile	View profile page	Yes
PATCH	/user/profile	Update profile	Yes
GET	/user/change-password	View password form	Yes
PUT	/user/change-password	Update password	Yes

Admin Authentication

Method	Endpoint	Description	Auth Required
--------	----------	-------------	---------------

GET	/admin/login	Admin login page	No
POST	/admin/login	Admin login	No
POST	/admin/logout	Admin logout	Yes

Categories Management

Method	Endpoint	Description	Auth Required
GET	/admin/categories	List all categories	Yes (Admin)
GET	/admin/categories/create	Show create form	Yes (Admin)
POST	/admin/categories	Create new category	Yes (Admin)
GET	/admin/categories/{id}	Show category details	Yes (Admin)
GET	/admin/categories/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/categories/{id}	Update category	Yes (Admin)
DELETE	/admin/categories/{id}	Delete category	Yes (Admin)
GET	/admin/categories/data/get	Get categories data (JSON)	Yes (Admin)
POST	/admin/categories/bulk-delete	Bulk delete categories	Yes (Admin)
POST	/admin/categories/bulk-status	Bulk update status	Yes (Admin)
POST	/admin/categories/check-name	Check name availability	Yes (Admin)
PATCH	/admin/categories/{id}/toggle-status	Toggle category status	Yes (Admin)

SubCategories Management

Method	Endpoint	Description	Auth Required
GET	/admin/subcategories	List all subcategories	Yes (Admin)
GET	/admin/subcategories/create	Show create form	Yes (Admin)
POST	/admin/subcategories	Create new subcategory	Yes (Admin)
GET	/admin/subcategories/{id}	Show subcategory details	Yes (Admin)
GET	/admin/subcategories/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/subcategories/{id}	Update subcategory	Yes (Admin)
DELETE	/admin/subcategories/{id}	Delete subcategory	Yes (Admin)
GET	/admin/subcategories/data/get	Get subcategories data (JSON)	Yes (Admin)
POST	/admin/subcategories/bulk-delete	Bulk delete subcategories	Yes (Admin)
POST	/admin/subcategories/bulk-status	Bulk update status	Yes (Admin)
POST	/admin/subcategories/check-name	Check name availability	Yes (Admin)
PATCH	/admin/subcategories/{id}/toggle-status	Toggle subcategory status	Yes (Admin)

Menu Management

Method	Endpoint	Description	Auth Required
GET	/admin/menu	List all menu items	Yes (Admin)
GET	/admin/menu/create	Show create form	Yes (Admin)
POST	/admin/menu	Create new menu item	Yes (Admin)
GET	/admin/menu/{id}	Show menu item details	Yes (Admin)
GET	/admin/menu/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/menu/{id}	Update menu item	Yes (Admin)
DELETE	/admin/menu/{id}	Delete menu item	Yes (Admin)
GET	/admin/menu/data/get	Get menu data (JSON)	Yes (Admin)
POST	/admin/menu/bulk-delete	Bulk delete menu items	Yes (Admin)
POST	/admin/menu/bulk-status	Bulk update status	Yes (Admin)
PATCH	/admin/menu/{id}/toggle-status	Toggle menu item status	Yes (Admin)
GET	/menu	Public menu page	No

Order Management

Method	Endpoint	Description	Auth Required
GET	/admin/orders	List all orders	Yes (Admin)
GET	/admin/orders/create	Show create form	Yes (Admin)
POST	/admin/orders	Create new order	Yes (Admin)
GET	/admin/orders/{id}	Show order details	Yes (Admin)
GET	/admin/orders/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/orders/{id}	Update order	Yes (Admin)

DELETE	/admin/orders/{id}	Delete order	Yes (Admin)
GET	/admin/orders/data/get	Get orders data (JSON)	Yes (Admin)
GET	/admin/orders/pending	List pending orders	Yes (Admin)
GET	/admin/orders/completed	List completed orders	Yes (Admin)
POST	/admin/orders/bulk-delete	Bulk delete orders	Yes (Admin)
POST	/admin/orders/bulk-status	Bulk update status	Yes (Admin)
PATCH	/admin/orders/{id}/status	Update order status	Yes (Admin)
GET	/user/orders	User's order history	Yes (User)

Reservation Management

Method	Endpoint	Description	Auth Required
GET	/admin/reservations	List all reservations	Yes (Admin)
GET	/admin/reservations/create	Show create form	Yes (Admin)
POST	/admin/reservations	Create new reservation	Yes (Admin)
GET	/admin/reservations/{id}	Show reservation details	Yes (Admin)
GET	/admin/reservations/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/reservations/{id}	Update reservation	Yes (Admin)
DELETE	/admin/reservations/{id}	Delete reservation	Yes (Admin)
GET	/admin/reservations/data/get	Get reservations data (JSON)	Yes (Admin)
GET	/admin/reservations/pending	List pending reservations	Yes (Admin)
GET	/admin/reservations/completed	List completed reservations	Yes (Admin)
POST	/admin/reservations/bulk-delete	Bulk delete reservations	Yes (Admin)
POST	/admin/reservations/bulk-status	Bulk update status	Yes (Admin)
PATCH	/admin/reservations/{id}/status	Update reservation status	Yes (Admin)

GET	/user/reservations	User's reservation history	Yes (User)
POST	/reserve-table	Create public reservation	No

Table Management

Method	Endpoint	Description	Auth Required
GET	/admin/tables	List all tables	Yes (Admin)
GET	/admin/tables/create	Show create form	Yes (Admin)
POST	/admin/tables	Create new table	Yes (Admin)
GET	/admin/tables/{id}	Show table details	Yes (Admin)
GET	/admin/tables/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/tables/{id}	Update table	Yes (Admin)
DELETE	/admin/tables/{id}	Delete table	Yes (Admin)
GET	/admin/tables/data/get	Get tables data (JSON)	Yes (Admin)
POST	/admin/tables/bulk-delete	Bulk delete tables	Yes (Admin)
POST	/admin/tables/bulk-status	Bulk update status	Yes (Admin)
PATCH	/admin/tables/{id}/toggle-status	Toggle table status	Yes (Admin)

Table Management

Method	Endpoint	Description	Auth Required
GET	/admin/customers	List all customers	Yes (Admin)
GET	/admin/customers/create	Show create form	Yes (Admin)
POST	/admin/customers	Create new customer	Yes (Admin)

GET	/admin/customers/{id}	Show customer details	Yes (Admin)
GET	/admin/customers/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/customers/{id}	Update customer	Yes (Admin)
DELETE	/admin/customers/{id}	Delete customer	Yes (Admin)
GET	/admin/customers/data/get	Get customers data (JSON)	Yes (Admin)
POST	/admin/customers/bulk-delete	Bulk delete customers	Yes (Admin)
POST	/admin/customers/bulk-status	Bulk update status	Yes (Admin)
PATCH	/admin/customers/{id}/toggle-status	Toggle customer status	Yes (Admin)

User Management (Admin)

Method	Endpoint	Description	Auth Required
GET	/admin/users	List all users	Yes (Admin)
GET	/admin/users/create	Show create form	Yes (Admin)
POST	/admin/users	Create new user	Yes (Admin)
GET	/admin/users/{id}	Show user details	Yes (Admin)
GET	/admin/users/{id}/edit	Show edit form	Yes (Admin)
PUT	/admin/users/{id}	Update user	Yes (Admin)
DELETE	/admin/users/{id}	Delete user	Yes (Admin)
GET	/admin/users/data/get	Get users data (JSON)	Yes (Admin)
POST	/admin/users/bulk-delete	Bulk delete users	Yes (Admin)
POST	/admin/users/bulk-status	Bulk update status	Yes (Admin)
PATCH	/admin/users/{id}/toggle-status	Toggle user status	Yes (Admin)

Dialogflow Integration

Method	Endpoint	Description	Auth Required
POST	/api/dialogflow/webhook	Dialogflow webhook handler	No
POST	/dialogflow/webhook	Dialogflow webhook (web route)	No
GET	/admin/dialogflow/export	Export all Dialogflow data	Yes (Admin)
GET	/admin/dialogflow/export/menu-categories	Export menu categories	Yes (Admin)
GET	/admin/dialogflow/export/menu-items	Export menu items	Yes (Admin)
GET	/admin/dialogflow/export/faq-topics	Export FAQ topics	Yes (Admin)
GET	/admin/dialogflow/export/quantities	Export quantities	Yes (Admin)
GET	/admin/dialogflow/export/status-types	Export status types	Yes (Admin)
GET	/admin/dialogflow/export/datetime	Export datetime patterns	Yes (Admin)
GET	/admin/dialogflow/export/party-size	Export party size options	Yes (Admin)
GET	/admin/dialogflow/export/names	Export name patterns	Yes (Admin)
GET	/admin/dialogflow/export/emails	Export email patterns	Yes (Admin)
GET	/admin/dialogflow/export/phones	Export phone patterns	Yes (Admin)

Public Pages

Method	Endpoint	Description	Auth Required
GET	/	Home page	No
GET	/about	About page	No
GET	/contact	Contact page	No

POST	/contact	Submit contact form	No
------	----------	---------------------	----

18.4 Dialogflow Webhook Integration

The webhook endpoint “/api/webhook/dialogflow” is used by Dialogflow to:

- Send user intent data (e.g., "place order", "check status")
- Receive dynamic responses based on backend logic
- Return JSON-formatted chatbot replies in real time

Security: The webhook uses CSRF protection and token validation to ensure only authorized requests are processed.

18.5 API Tools Used

- **Postman** – For testing and debugging API endpoints
- **Laravel Routes + Controllers** – For API handling
- **Middleware** – For role-based access control (admin vs. user)
- **Validation** – Built-in Laravel validation for form inputs and requests

19. Security Considerations

Security is a critical component of any web-based application, especially those involving user data, order processing, and administrative controls. The EatzaI system incorporates multiple security measures across its architecture to ensure data protection, access control, and secure communication between system components.

19.1 User Authentication & Authorization

- **Login Protection:** Both user and admin logins are protected using Laravel's built-in authentication mechanism. Passwords are securely hashed using **bcrypt**.
- **Role-Based Access:** Access to specific routes and dashboard sections is restricted based on user roles (admin or user). This logic is implemented using conditional checks in routes and middleware.
- **Session Management:** Laravel session tokens are used to manage authenticated sessions. Sessions expire after inactivity for a configurable period.

19.2 Input Validation & Sanitization

- All form inputs are validated on both frontend and backend using Laravel's validation system.
- User-provided data is sanitized to prevent SQL injection and malformed input.
- APIs reject invalid or malicious data using defined validation rules.

19.3 API Security

- All API endpoints use **token-based or session-based** authentication to ensure only verified users can access system resources.
- **Rate limiting** and request throttling can be configured to prevent abuse of public endpoints.
- Webhook endpoints used by Dialogflow are restricted to verified sources and are protected against unauthorized access.

19.4 Database Security

- The database credentials are stored securely in the `.env` file, which is not accessible publicly.
- Direct SQL access is avoided; instead, all data access is handled via **Laravel's Eloquent ORM**, which automatically guards against SQL injection.
- Foreign key constraints are used to maintain relational integrity between tables.

19.5 Chatbot Security

- The Dialogflow webhook validates and sanitizes all incoming requests before processing.
- Chatbot responses are generated based on verified backend data, preventing injection of misleading or untrusted content.

19.6 Hosting & Server Considerations

- The system is hosted using **InfinityFree**, a shared hosting provider with basic SSL support enabled.
- HTTPS (SSL) is used for encrypted communication between the browser and the server.
- Directory permissions and environment files are configured to restrict unauthorized access to server resources.

19.7 Error Handling and Logging

- Laravel handles exceptions gracefully and provides custom error messages without revealing sensitive system details.
- Error logs are stored securely on the server for review by administrators.

19.8 Future Improvements

- Implement **Two-Factor Authentication (2FA)** for admin accounts.
- Add **reCAPTCHA** for login and registration forms to prevent bot attacks.
- Transition to **OAuth or JWT authentication** for APIs when scaling beyond the current deployment.

The EatzaI system's security design ensures that sensitive data is protected, unauthorized access is prevented, and the integrity of restaurant operations remains intact across all user interactions.

20. Testing Plan and Test Cases

Effective testing is essential to ensure that the EatzAI system meets functional and non-functional requirements. The testing process validates chatbot behavior, portal functionality, data handling, and system reliability under various scenarios. This section outlines the testing strategy and a sample of test cases executed during the system development lifecycle.

20.1 Testing Strategy

The testing plan follows a **multi-phase approach**:

- **Unit Testing:** Verifying individual functions such as order processing, reservation creation, and API endpoints.
- **Integration Testing:** Testing interactions between the chatbot, Laravel backend, and database.
- **System Testing:** End-to-end testing of complete workflows across the user and admin portals.
- **User Acceptance Testing (UAT):** Final testing performed by users and stakeholders to validate the system meets business goals.

20.2 Testing Tools Used

Tool	Purpose
Postman	Testing RESTful APIs
Laravel Artisan	Running unit tests
Dialogflow Console	Intent testing and response validation
Browser Developer Tools	Frontend inspection and debugging
Manual Testing	Functional and UI/UX verification

20.3 Sample Test Cases

Test Case ID	Test Scenario	User Input / Trigger	Expected Bot Response	Status
TC01	Greet the user on chatbot open	"Hi" / "Hello" / "Hey"	"Hi! Welcome to EatzaI. How can I assist you today?"	Pass
TC02	Show available menu categories	"Show me the menu"	Displays categories: Main Course, Drinks, Desserts, etc.	Pass
TC03	Show items under a specific category	"What do you have in drinks?"	Lists drink items (e.g., Cola, Lemonade, Water, etc.)	Pass
TC04	Add item to order	"I want 2 burgers"	Confirms item added: "2 burgers have been added to your order."	Pass
TC05	Place final order	"Place my order" / "Checkout"	Confirms and shows order summary with total	Pass
TC06	Ask for order status	"Track my order"	Responds with current order status (e.g., "Being prepared")	Pass
TC07	Book a table via chatbot	"Book a table for 4 at 7 PM tonight"	Confirms reservation and provides a reservation ID	Pass
TC08	Ask a FAQ (e.g., hours, payment, location)	"What are your opening hours?"	"We are open daily from 10:00 AM to 10:00 PM."	Pass
TC09	Ask an unsupported question	"Can I update my email?"	"Sorry, I can't help with that. Please use the user portal."	Pass
TC10	Unrecognized input	"Can I fly in for dinner?"	Triggers fallback: "Sorry, I didn't understand that."	Pass

20.4 Bug Tracking and Resolution

All bugs discovered during testing were logged and tracked using a manual spreadsheet. Each issue was assigned severity and resolved based on priority. Regression testing was performed after bug fixes to confirm no new issues were introduced.

20.5 Test Coverage

The EatzaI system achieved high coverage across:

- Chatbot workflows (ordering, FAQs, reservation)
- Portal functionalities (authentication, CRUD operations)
- Backend API endpoints
- Input validation and error handling

20.6 Conclusion

Testing ensured that the EatzaI application is reliable, user-friendly, and performs its key functions accurately. The system successfully passed all defined test cases and is ready for deployment and real-world usage.

21. Development Methodology

The development of the EatzaI system followed a **hybrid software development methodology**, combining elements of both the **Waterfall** and **Agile** approaches. This strategy was chosen to balance structured planning and flexible execution, ensuring clear documentation while allowing iterative enhancements based on feedback and testing.

21.1 Waterfall Phase (Planning and Design)

In the early stages of the project, a **Waterfall approach** was applied to ensure proper documentation, requirement analysis, and design. This phase included:

- **Requirement Gathering:** Defining functional and non-functional requirements based on restaurant workflow analysis.
- **System Design:** Creating use case diagrams, ERDs, architecture diagrams, class diagrams, and UI wireframes.
- **SRS and Design Documentation:** Writing comprehensive documents to guide development and stakeholder understanding.

This structured planning allowed the team to clearly define project boundaries, goals, and data models before beginning implementation.

21.2 Agile Phase (Development and Testing)

Once development began, an **Agile-inspired, iterative approach** was adopted to ensure flexibility, faster feature delivery, and responsiveness to feedback.

- **Sprint-Based Development:** Features were broken into short, manageable sprints (e.g., user portal, admin portal, chatbot integration).
- **Continuous Testing:** Unit tests and integration tests were performed during and after each sprint to verify functionality.

- **Incremental Enhancements:** Features were released progressively (starting with order placement and reservation, followed by menu browsing, order tracking, and FAQs).
- **Feedback Loops:** Regular testing by the development team and supervisor guided adjustments to chatbot logic, portal behavior, and system usability.

21.3 Tools Used in Development

Phase	Tool/Technology
Requirement Analysis	Microsoft Word, Google Docs
Diagramming	draw.io, Lucidchart
Backend Development	Laravel, PHPStorm/VS Code, XAMPP
Frontend Development	HTML, CSS, Bootstrap, JavaScript
Chatbot Integration	Google Dialogflow, Dialogflow Webhook
Testing	Postman, Laravel Artisan test, Dialogflow Console
Version Control	Git, GitHub

21.4 Advantages of Hybrid Approach

- Ensures **comprehensive documentation** and traceability of system decisions (Waterfall).
- Provides **flexibility and faster iteration** during coding and testing (Agile).
- Supports better **risk management** and **scope control**.
- Enables **continuous feedback** and improvements without disrupting overall system flow.

21.5 Summary

By combining Waterfall's structure with Agile's flexibility, the development team was able to deliver a feature-rich, well-documented, and fully functional restaurant chatbot system that aligns with real-world

operational needs. The hybrid methodology ensured that both academic requirements and user expectations were successfully met.

22. Work Plan / Schedule

An organized work plan was followed to ensure timely and structured development of the EatzAI system. The entire project was broken down into well-defined phases, spanning from requirements gathering to final testing, deployment, and documentation. Each milestone included technical development and skill enhancement, particularly with tools like Laravel and Google Dialogflow.

22.1 Project Phases and Timeline

Phase	Task Description	Start Date	End Date	Status
Phase 1: Initiation	Project idea selection, scope definition, preliminary research	01-Nov-2024	10-Nov-2024	Completed
Phase 2: Requirements Gathering	Functional & non-functional requirements, chatbot use case drafting	11-Nov-2024	20-Nov-2024	Completed
Phase 3: Documentation	SRS document preparation, system analysis diagrams	21-Nov-2024	10-Dec-2024	Completed
Phase 4: UI/UX & Design	Wireframes for user/admin portal, chatbot layout, ERD, class, sequence diagrams	11-Dec-2024	25-Dec-2024	Completed
Phase 5: Backend Setup	Laravel project initialization, DB schema, role-based login system	26-Dec-2024	10-Jan-2025	Completed
Phase 6: Chatbot Development	Google Dialogflow agent setup, intent/entity training, webhook logic (Laravel)	11-Jan-2025	25-Jan-2025	Completed
Phase 7: Portal Development	Admin & User portals: login, CRUD for orders, reservations, menu, FAQs	26-Jan-2025	15-Feb-2025	Completed

Phase 8: API Integration	REST API development & linking chatbot with Laravel backend via webhook	16-Feb-2025	29-Feb-2025	Completed
Phase 9: Testing & Debugging	Unit testing, chatbot test cases, Laravel route testing, bug fixes	01-Mar-2025	15-Mar-2025	Completed
Phase 10: Deployment	Hosting live version on InfinityFree, chatbot integration, SSL, domain testing	16-Mar-2025	25-Mar-2025	Completed
Phase 11: Final Report Writing	Compilation of SRS, design document, implementation, testing plans	26-Mar-2025	10-Apr-2025	Completed
Phase 12: Final Review, Learning & Submission	Supervisor feedback, formatting, personal learning reflections (e.g., Dialogflow agent training, webhook integration, testing challenges)	11-Apr-2025	25-Apr-2025	Completed

22.2 Tools Used for Project Planning and Execution

Tool / Platform	Purpose
Microsoft Excel / Gantt Chart	Task tracking and timeline management
Google Docs / Word	Report writing and document collaboration
draw.io / Lucidchart	System design diagrams (ERD, sequence, class)
Laravel Framework	Backend logic and API development
Google Dialogflow	Chatbot development, intent training, webhook setup
Postman	API testing and integration verification
Git & GitHub	Version control and change tracking
InfinityFree Hosting	Live deployment of Laravel application

22.3 Learning Reflection

During this project, one of the most significant technical learning curves was mastering **Google Dialogflow**, including:

- Designing intent flows and training phrases
- Building custom entities
- Managing conversation context
- Integrating with Laravel through secure webhook fulfilment

In parallel, building REST APIs, database schema management, and implementing a **custom role-based login system** in Laravel enhanced backend development expertise. These practical skills, gained while solving real restaurant workflow problems, were essential to delivering a functional and deployable chatbot-based solution.

23. References

This section lists all resources consulted or referenced during the development of the EatzaI system, including academic literature, technical documentation, development tools, and real-world website inspiration. Proper referencing demonstrates ethical project practices and acknowledges sources that guided system design and implementation.

23.1 Academic & Technical References

1. Sommerville, Ian. *Software Engineering*, 10th Edition. Pearson Education, 2015.
2. Pressman, Roger S. *Software Engineering: A Practitioner's Approach*, 8th Edition. McGraw-Hill, 2014.
3. IEEE. *IEEE Standard for Software Requirements Specifications*, IEEE Std 830-1998.

23.2 Online Documentation & Tutorials

1. [Laravel Documentation](#) – Official documentation for Laravel framework used to build backend APIs and portals.
2. [Google Dialogflow Documentation](#) – For chatbot intent modeling, webhook integration, and NLP capabilities.
3. [Bootstrap Documentation](#) – For creating responsive UI layouts.
4. [W3Schools](#) – For HTML5, CSS3, PHP, and JavaScript syntax reference.
5. [MySQL Reference Manual](#) – For database modeling and query structure.
6. [Postman Learning Center](#) – Used to test Laravel APIs and webhook integration.
7. [draw.io](#) – Tool for creating system diagrams like ERD, class, and sequence diagrams.
8. [Stack Overflow](#) – Peer support and troubleshooting for development issues.

23.3 Hosting & Deployment

1. [InfinityFree](#) – Free web hosting provider used to deploy the live Laravel application.
2. [phpMyAdmin](#) – Tool used for managing MySQL database operations during development and testing.

23.4 AI & NLP Resources

1. [Dialogflow Tutorials](#) – For building and testing conversational agents.

23.5 Real-World Reference Website

1. [FoodsInn Restaurant](#) – Used as a real-world reference to understand food menu structuring, reservation systems, and user experience design for restaurant platforms.

24. Appendices

This section contains supplementary materials that support the main content of the report. These items provide deeper insight into the system design, implementation, and testing. Appendices are included for completeness and to allow reviewers, developers, or future maintainers to refer to technical details without crowding the main sections of the report.

24.1 Sample Dialogflow Intents

A selection of configured intents used in the EatzaAI chatbot system:

Intent Name	Purpose	Fulfilled by Webhook
menu.show	Lists all menu categories	Yes
order.add	Adds item(s) to order	Yes
order.status	Tracks order status	Yes
reservation.create	Creates a new table reservation	Yes
faq.inquiry	Answers frequently asked questions	No
Default Fallback	Handles unrecognized queries	No

24.2 Database Schema (SQL Export)

- A full SQL export of the database structure, including:
 - users, orders, order_items, menu_items, categories, reservations, and faqs tables.
- Foreign key relationships
- Sample dummy data used for testing

(Attach/export as database_schema.sql or include in printed annexe)

24.3 Test Case Matrix (Chatbot)

Test Case ID	Customer Action	User Input	Intent Detected	Expected Response	Result
TC01	Make Reservation	Book a table for 4 people	reservation.create	Reservation confirmation	Pass
TC02	Make Reservation	I need a reservation for tonight	reservation.create	Reservation confirmation	Pass
TC03	Make Reservation	Reserve a table for 2 at 7 PM	reservation.create	Reservation confirmation	Pass
TC04	Check Reservation	Show my reservation details	reservation.details	Reservation details display	Pass
TC05	Check Reservation	What's my reservation status?	reservation.status	Status confirmation	Pass
TC06	Place Order	Start a new order	new.order	New order session started	Pass
TC07	Place Order	Add pizza to my order	order.add	Item added confirmation	Pass
TC08	Place Order	Add burger and fries	order.add	Items added confirmation	Pass
TC09	Place Order	Remove pizza from my order	order.remove	Item removed confirmation	Pass
TC10	Place Order	Checkout my order	checkout.order	Order summary and confirmation	Pass
TC11	Track Order	Track my order	track.order	Order status message	Pass
TC12	Track Order	Where is my order?	track.order	Order status message	Pass
TC13	Track Order	What's the status of my order?	track.order	Order status message	Pass

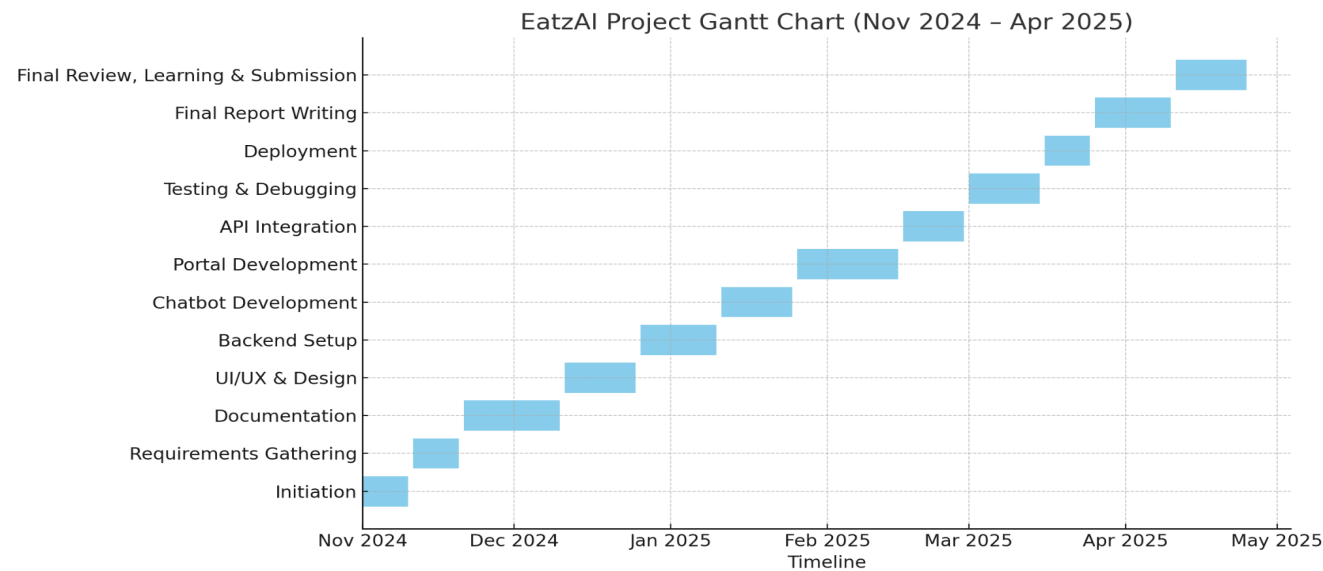
24.4 Screenshots

- Admin Dashboard
- User Profile View
- Chatbot Interface with sample conversation
- Menu Management UI
- Reservation Logs and History

(Screenshots to be labelled and inserted with figure numbers such as Figure A1, A2, etc.)

24.5 Gantt Chart

A visual timeline of project phases from November 2024 to April 2025.



(Attach Gantt chart image or link to Google Sheets/Excel file)

